

Ubicación de Valores por Defecto en el Sistema GLM

1. FlashCellIdentifier (flash_cell_identification.py)

Líneas 20-22:

```
python
```

```
def __init__(self, eps=0.05, min_samples=5, use_time_weight=True):
```

Valores por defecto:

- `eps=0.05` - Distancia máxima DBSCAN
 - `min_samples=5` - Número mínimo de muestras por cluster
 - `use_time_weight=True` - Usar peso temporal en clustering
-

2. ImprovedFlashCellTracker (improved_flash_cell_tracking.py)

Líneas 15-18:

```
python
```

```
def __init__(self, max_distance_km=30, max_speed_kmh=120,  
              intensity_weight=0.3, size_weight=0.2, shape_weight=0.1,  
              prediction_weight=0.4, overlap_threshold=0.1):
```

Línea 38:

```
python
```

```
self.time_delta_minutes = 10 # Ventana temporal típica
```

Líneas 41-42:

```
python
```

```
self.adaptive_thresholds = True  
self.min_track_length = 2
```

Valores por defecto:

- `max_distance_km=30` - Distancia máxima para asociación
- `max_speed_kmh=120` - Velocidad máxima realista
- `intensity_weight=0.3` - Peso similitud intensidad

- `size_weight=0.2` - Peso similitud tamaño
 - `shape_weight=0.1` - Peso similitud forma
 - `prediction_weight=0.4` - Peso predicción posición
 - `overlap_threshold=0.1` - Umbral solapamiento
 - `time_delta_minutes=10` - Ventana temporal
 - `adaptive_thresholds=True` - Umbrales adaptativos
 - `min_track_length=2` - Longitud mínima de track
-

3. SmartImprovedFlashCellNowcaster (improved_flash_cell_nowcasting.py)

Líneas 17-21:

python

```
def __init__(self, forecast_minutes=20, min_history_points=2,  
             ensemble_models=True, uncertainty_quantification=True,  
             training_data_dir=None):
```

Líneas 35-36:

python

```
self.min_training_samples = 50 # Mínimo para entrenar RF
```

Líneas 43-47:

python

```
self.velocity_stats = {  
    'mean_speed': 25.0,  
    'std_speed': 15.0,  
    'max_speed': 80.0  
}
```

Líneas 76-82 (Random Forest):

python

```
self.models['rf'] = RandomForestRegressor(  
    n_estimators=100,  
    max_depth=10,  
    min_samples_split=5,  
    min_samples_leaf=2,  
    random_state=42,  
    n_jobs=-1  
)
```

Valores por defecto:

- `forecast_minutes=20` - Tiempo de pronóstico
 - `min_history_points=2` - Puntos mínimos historial
 - `ensemble_models=True` - Usar ensemble
 - `uncertainty_quantification=True` - Calcular incertidumbre
 - `training_data_dir=None` - Directorio entrenamiento
 - `min_training_samples=50` - Muestras mínimas RF
 - `mean_speed=25.0` - Velocidad media estadística
 - `std_speed=15.0` - Desviación estándar velocidad
 - `max_speed=80.0` - Velocidad máxima estadística
 - `n_estimators=100` - Número de árboles RF
 - `max_depth=10` - Profundidad máxima RF
 - `min_samples_split=5` - Muestras mínimas split
 - `min_samples_leaf=2` - Muestras mínimas hoja
 - `random_state=42` - Semilla aleatoria
 - `n_jobs=-1` - Usar todos los cores
-

4. PerformanceVerificationSystem (verification_and_performance_metrics.py)

Línea 49:

python

```
def verify_prediction(self, prediction, actual_observation, tolerance_minutes=25):
```

Valores por defecto:

- `tolerance_minutes=25` - Tolerancia temporal para verificación
-

5. NowcastingEvaluator (nowcasting_evaluator.py)

Línea 26:

python

```
def calculate_prediction_metrics(self, actual_cells_df, predicted_cells_df,
                                position_tolerance_km=10.0):
```

Valores por defecto:

- `position_tolerance_km=10.0` - Tolerancia posicional para aciertos
-

6. EnhancedLightningVisualizer (enhanced_lightning_visualizer.py)

Líneas 23-27:

python

```
self.track_colors = [
    "#FF0000", "#00FF00", "#0000FF", "#FFFF00", "#FF00FF",
    "#00FFFF", "#FF8000", "#8000FF", "#0080FF", "#FF0080",
    "#80FF00", "#00FF80", "#FF8080", "#80FF80", "#8080FF",
    "#FFFF80", "#FF80FF", "#80FFFF", "#FF4000", "#40FF00"
]
```

Líneas 30-35:

python

```
self.confidence_levels = [
    ('error_90ci_km', 0.1, '90% confianza', '#FF0000'),
    ('error_80ci_km', 0.15, '80% confianza', '#FF4000'),
    ('error_60ci_km', 0.25, '60% confianza', '#FF8000'),
    ('error_40ci_km', 0.35, '40% confianza', '#FFFF00')
]
```

Valores por defecto:

- Lista de 20 colores para tracks
 - 4 niveles de confianza con opacidades específicas
-

7. Script run_historical_analysis.sh

Líneas 20-21:

bash

```
echo " --window_minutes N      Tamaño de ventana temporal en minutos (default: 10)"
echo " --forecast_minutes N    Minutos hacia el futuro para forecast (default: 20)"
```

Líneas 26-27:

bash

```
echo " --eps N                  Parámetro eps para DBSCAN (default: 0.05)"
echo " --min_samples N         Parámetro min_samples para DBSCAN (default: 5)"
```

Valores por defecto indicados en ayuda:

- `window_minutes=10` - Ventana temporal
 - `forecast_minutes=20` - Tiempo pronóstico
 - `eps=0.05` - Parámetro DBSCAN
 - `min_samples=5` - Muestras mínimas DBSCAN
-

8. Valores Hardcodeados en Funciones

FlashCellIdentifier

Línea 140 (identify_cells):

python

```
small_eps = min(self.eps, 0.015) # Usar un valor máximo de 0.015
min_points = max(3, self.min_samples - 2) # Reducir el mínimo de puntos
```

Línea 216 (manual grouping):

python

```
connected = distances < 0.02 # 0.02 grados
```

ImprovedFlashCellTracker

Línea 218 (solve_assignment_problem):

python

```
if similarity > 0.2: # Umbral mínimo adaptativo
```

Líneas 231-236 (adapt_thresholds):

python

```
if n_cells > 20: # Muchas celdas, ser más restrictivo
    self.max_distance_km = min(self.max_distance_km, 25)
elif n_cells < 5: # Pocas celdas, ser más permisivo
    self.max_distance_km = max(self.max_distance_km, 35)
```

Línea 371 (track cleanup):

python

```
cutoff_time = timestamp - pd.Timedelta(minutes=30) # 30 minutos
```

SmartImprovedFlashCellNowcaster

Línea 118 (re-training frequency):

python

```
elif self.rf_trained and len(self.training_data) % 25 == 0: # Cada 25 nuevas muestras
```

Líneas 393-394 (physics constraints):

python

```
max_intensity = 1000
pred_area = min(pred_area, 10000) # Máximo 10000 km2
```

9. Valores de Configuración de Directorio

run_historical_analysis.sh

Líneas 8-12:

bash

```
NOWCASTING_DIR="/home/matias/nowcasting"
DATA_DIR="${NOWCASTING_DIR}/data/raw"
OUTPUT_DIR="${NOWCASTING_DIR}/outputs/historical"
LOG_DIR="${NOWCASTING_DIR}/logs"
```

10. Constantes Matemáticas y Físicas

Múltiples archivos - Radio de la Tierra:

python

```
R = 6371.0 # Radio de La Tierra en km
```

Conversión grados a km:

python

```
area_km2 = polygon.area * 111 * 111 # Aproximación área en km²
lat_radius = radius_km / 111.0 # Conversión
```

Resumen de Ubicaciones por Archivo:

Archivo	Líneas con Defaults	Cantidad de Parámetros
flash_cell_identification.py	20-22, 140, 216	5 parámetros
improved_flash_cell_tracking.py	15-18, 38, 41-42, 218, 231-236, 371	12 parámetros
improved_flash_cell_nowcasting.py	17-21, 35-36, 43-47, 76-82, 118, 393-394	15 parámetros
verification_and_performance_metrics.py	49	1 parámetro
nowcasting_evaluator.py	26	1 parámetro
enhanced_lightning_visualizer.py	23-27, 30-35	24 colores + 4 niveles
run_historical_analysis.sh	8-12, 20-21, 26-27	8 parámetros

Total: 66+ parámetros configurables distribuidos en 7 archivos